ADF Analyzer v10 — LOGIC Reference

This file documents the core scoring, thresholds and detection logic implemented across the analyzer and dashboard. It is intended to be an authoritative, developer-oriented reference you can link to in the UI or README.

Files that implement logic

- `adf_dashboard.py` — dashboard rendering, health gauge and metric tiles. Key functions:
  - `render_enhanced_metrics()` — derives displayed metrics and health score summary tiles.
  - `render_health_gauge()` — renders the health gauge and contains the canonical thresholds used for status labels.
  - Impact-level visuals and counts are calculated from the `impact`/`Impact` data frame.
- `adf_analyzer_v10_excel_enhancements.py` — analysis result processing and scoring for the exported Excel report.
  - `_calculate_quality_score()` — produces the consolidated "Quality Score" used in reports.
  - Formatting helpers generate severity/impact color-coding and produce the `CircularDependencies`, `Orphaned*` sheets.

Health score (Dashboard)

Purpose: provide a quick, 0–100 factory-level health indicator based on orphaned pipelines relative to total pipelines.

Formula (canonical):

- If pipelines > 0:
  health_score = int((1 - orphaned / pipelines) * 100)
- Else (no pipelines):
  health_score = 100

Notes:

- `orphaned` is taken from the `Orphaned Pipelines` metric (sheet `OrphanedPipelines` / `Orphaned_Pipelines`).
- This is the value used by the dashboard gauge (`render_health_gauge`) and by the top-row Health tile.

Thresholds / status mapping used in the gauge:

- health_score >= 90 → status: "Excellent" (green)
- 75 <= health_score < 90 → status: "Good" (blue)
- 60 <= health_score < 75 → status: "Fair" (yellow)
- health_score < 60 → status: "Needs Attention" (red)

Delta/visual reference: the gauge uses a reference of 80 for delta display.

Example:

- pipelines = 20, orphaned = 2 → health_score = int((1 - 2/20)*100) = 90 → Excellent
- pipelines = 10, orphaned = 3 → health_score = int((1 - 3/10)*100) = 70 → Fair

Quality score (Analyzer Excel)

Purpose: consolidated quality score (0–100) produced for the workbook by `_calculate_quality_score()`.

Calculation steps (starting from 100):

1. Deduct for circular dependencies:
    - Deduction = 10 points per cycle, capped at 30 points (i.e., max deduction 30).
    - Implementation: `score -= min(circular_deps * 10, 30)`
2. Deduct for orphaned resources:
    - orphaned = sum of orphaned pipelines, orphaned datasets, orphaned linked services
    - orphan_percentage = (orphaned / max(total resources, 1)) * 100
    - Deduction = min(orphan_percentage, 20) (cap 20 points)
3. Deduct for broken triggers:
    - Deduction = 5 points per broken trigger, capped at 15 points
    - Implementation: `score -= min(broken_triggers * 5, 15)`
4. Final clamp: result is clamped to [0, 100] and returned as integer.

Example:

- 2 circular cycles (20 pts), orphan_percentage 8% (8 pts), 1 broken trigger (5 pts) → score = 100 - 20 - 8 - 5 = 67

Files & locations:

- See `_calculate_quality_score()` in `adf_analyzer_v10_excel_enhancements.py` for the exact code.

Circular dependency detection

Purpose: detect resource cycles (pipelines/activities) that can cause infinite loops.

Algorithm and behavior:

- DFS traversal over the dependency graph (edges commonly come from `dependsOn`/activity references).
- When a back-edge to a node on the recursion stack is found, a cycle is extracted from the current path.
- Dedupe: cycles are canonicalized (rotated to a canonical smallest representation) to prevent duplicates detected from different start nodes.
- Output: `CircularDependencies` sheet with rows describing each cycle (Type, Cycle path, Length, Severity, Impact, Recommendation).

Severity: Typically cycles are marked as `CRITICAL` (production blocker) or `HIGH`. The analyzer/formatters color-code these in the report.

Action policy:

- Circular dependencies are considered CRITICAL: the README and formatting code treat any cycle as a production blocker and recommend immediate remediation.

Orphaned resources detection

Purpose: detect resources that are not referenced by others (e.g., pipelines not triggered or datasets not used). These indicate potential dead code or cleanup opportunities.

Detection sources:

- Sheets named `OrphanedPipelines`, `OrphanedDatasets`, `Orphaned_LinkedServices` (or similar) are produced during analysis.
- Orphans are counted and used in both the dashboard metrics and the quality score deduction.

Contribution to scores:

- Orphaned counts contribute to the dashboard Health score (orphaned/pipelines fraction).
- Orphaned resources reduce the Quality Score proportionally (via orphan percentage, capped at 20 points).

Impact levels & severity

- Impact levels (CRITICAL / HIGH / MEDIUM / LOW) appear in the `impact_analysis` results and are used to:
  - Color charts and metric badges in the dashboard (colors: CRITICAL = red, HIGH = orange, MEDIUM = yellow, LOW = green).
  - Count impact levels for a quick triage (dashboard shows counts of CRITICAL / HIGH / MEDIUM / LOW items).

Implementation notes:

- If no impact data exists, the dashboard default sets `Impact` to `LOW` for display purposes.
- Sorting/order respects `impact_order = {'CRITICAL': 0, 'HIGH': 1, 'MEDIUM': 2, 'LOW': 3}` in the analyzer code.

Other small rules & helpers

- Metric fallbacks: the dashboard tries to read metrics from a `Summary` sheet. If a metric is missing it falls back to counting rows on one of several candidate sheets (e.g., `ImpactAnalysis`, `PipelineAnalysis`, `Pipelines` for `Pipelines`).
- Activity lineage: static vs dynamic source/sink classification is done by scanning `SourceTable`/`SinkTable` values for patterns like `@dataset`, `@{`, `pipeline()` or `activity()` to detect parameterized/dynamic references.
- Impact visualization colors are defined in the dashboard near `render_impact` and `render_analysis` helpers.

Triggers counting (special rule)

- Preferred source: `Triggers` sheet (one row per trigger). When present, the dashboard uses the row count from this sheet as the canonical trigger count.
- Fallback: `TriggerDetails` — this sheet contains trigger usage/occurrence rows and can have multiple rows per trigger. If `Triggers` is absent the code will count unique trigger names from the `TriggerDetails` `Trigger`/`Name` column (if present) and only fall back to the raw `TriggerDetails` row count when no clear name column exists.
- Reason: counting `TriggerDetails` rows directly may over-count triggers because a single trigger can appear multiple times (different schedules, pipelines, or environments). The special-case rule prevents inflated trigger counts.

Edge cases and assumptions

- Division by zero: health score uses `pipelines > 0` guard; otherwise health_score defaults to 100.
- Resource totals used for orphan-percentage computation use `max(total_resources, 1)` to avoid division by zero.
- Deduction caps are intentionally conservative to avoid a single failure mode pushing the quality score to zero instantly.

Where to change behavior

- Health score thresholds / formula: edit `render_health_gauge()` and `render_enhanced_metrics()` in `adf_dashboard.py`.
- Quality score weights & caps: edit `_calculate_quality_score()` in `adf_analyzer_v10_excel_enhancements.py`.
- Circular detection algorithm: search for `detect_circular_dependencies` and the DFS implementation in the analyzer module(s); canonicalization/deduplication is implemented on the detected cycles before reporting.

Suggested follow-ups

- Consider unifying score formulas and single-source-of-truth functions to avoid subtle divergence between dashboard and analyzer (health score was harmonized in the dashboard codebase but the analyzer quality score remains separate).
- Add unit tests for scoring functions to lock behavior (happy path + edge cases: zero pipelines, many cycles, many orphans).

If you want, I can:

- Add a short link to `LOGIC.md` in the top of the README.
- Extract the health/quality scoring logic into a small helper module and add unit tests.